手写小GPT学习笔记 | Day14 周测复盘

Day14是**Attention模块专项周测**，复盘Day8-Day13全部核心知识点，通过理论自测、代码复现、实战调试三大环节，验收学习成果、查漏补缺，彻底吃透Transformer注意力模块全流程，为后续搭建完整GPT模型筑牢根基。

一、Day14 周测目标

- **理论通关**：默写核心原理、公式、架构逻辑，理清各组件分工与关联
- **代码复现**：不依赖笔记，独立手写完整Attention模块，实现无Bug运行
- **问题排查**：定位高频报错原因，掌握调试技巧，巩固避坑要点
- **知识闭环**：梳理组件串联逻辑，总结学习心得，明确后续进阶方向

二、理论自测题（闭卷作答）

先完成理论自测，检验原理掌握程度，答案附在文末，错题重点标注复盘。

1. 基础概念题

1. Scaled Dot-Product Attention 中，为什么要除以 $\sqrt{d_k}$？不除会有什么后果？
2. Padding Mask 和 Causal Mask 的作用分别是什么？适用场景有何区别？
3. Multi-Head Attention 相比单头注意力，优势在哪里？核心计算流程是什么？
4. Pre-LN 和 Post-LN 架构的区别？为什么GPT选择Pre-LN？
5. 残差连接的核心作用是什么？为什么残差分支不能加Dropout？
6. FFN网络的设计逻辑是什么？为什么要先升维再降维？激活函数为什么选GELU？

2. 维度计算题

已知：batch_size=2，seq_len=10，d_model=512，num_heads=8，求以下张量shape：

1. QKV投影后的shape（未拆分多头）
2. 拆分多头后的shape
3. 注意力分数attn_scores的shape
4. FFN升维后的特征shape

三、代码周测：独立复现完整Attention模块

测试要求：不翻看前期笔记，从零手写以下组件，最终实现完整Transformer Block前向传播，验证维度守恒。

测试清单（必写组件）

- ✅ 缩放点积注意力（Scaled Dot-Product Attention）
- ✅ 两种掩码实现（Padding Mask + Causal Mask）
- ✅ 多头注意力（Multi-Head Attention）
- ✅ 前馈网络（FFN）
- ✅ 带残差、LayerNorm的完整Transformer Block
- ✅ 测试脚本（验证输入输出维度）

周测标准代码（参考答案，写完对照）

整合所有组件的精简版标准代码，适配自测复现，注释精简，方便对照纠错。

Day14 周测标准代码

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import os

os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# 1. 掩码工具
def create_padding_mask(seq, pad_idx=0):
    return (seq == pad_idx).unsqueeze(1).unsqueeze(2)

def create_causal_mask(seq_len):
    return torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool)).unsqueeze(0).unsqueeze(0)

# 2. 缩放点积注意力
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
        if mask is not None:
```

```python
            scores = scores.masked_fill(mask == False, -1e9)
        attn = F.softmax(scores, dim=-1)
        attn = self.dropout(attn)
        return torch.matmul(attn, v), attn

# 3. 多头注意力
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)
    def split_heads(self, x, batch_size):
        return x.view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
    def combine_heads(self, x, batch_size):
        return x.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)
    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)
        q_proj = self.split_heads(self.w_q(q), batch_size)
        k_proj = self.split_heads(self.w_k(k), batch_size)
        v_proj = self.split_heads(self.w_v(v), batch_size)
        attn_out, _ = self.attn(q_proj, k_proj, v_proj, mask)
        out = self.combine_heads(attn_out, batch_size)
        return self.dropout(self.w_o(out))

# 4. FFN前馈网络
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion=4):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_model*expansion)
        self.linear2 = nn.Linear(d_model*expansion, d_model)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(dropout)
    def forward(self, x):
        return self.linear2(self.dropout(self.gelu(self.linear1(x))))

# 5. 完整Transformer Block
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
```

```python
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ffn = FeedForwardNetwork(d_model, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)
    def forward(self, x, mask=None):
        # 注意力子层
        residual = x
        x = self.norm1(x)
        x = residual + self.dropout(self.attn(x, x, x, mask))
        # FFN子层
        residual = x
        x = self.norm2(x)
        x = residual + self.dropout(self.ffn(x))
        return x
```

四、理论自测答案+错题复盘

1. 基础概念题答案

1. **缩放原因**：防止qk点积后数值过大，导致softmax梯度消失；不除会让注意力分数极端化，模型无法学习有效权重。
2. **Padding Mask**：屏蔽填充符，避免模型学习无效占位符，适用于编码器+解码器；**Causal Mask**：屏蔽未来信息，保证解码器自回归特性，仅用于解码器自注意力。
3. **多头优势**：捕捉不同粒度、不同位置的语义依赖，表达能力更强；流程：QKV投影→拆分多头→缩放点积注意力→拼接多头→输出投影。
4. **Pre-LN**：先归一化再运算，梯度传播更稳定，深层不易消失；Post-LN：先运算再归一化，深层训练易震荡，GPT选Pre-LN提升训练稳定性。
5. **残差作用**：打通梯度高速通道，防止梯度消失，保留原始特征；残差加Dropout会破坏原始信息、阻断梯度通路，导致训练崩盘。
6. **FFN逻辑**：升维捕捉高维特征，降维回归模型维度；GELU是平滑激活函数，避免负区间梯度消失，适配深层网络。

2. 维度计算题答案

1. QKV投影后：[2,10,512]
2. 拆分多头后：[2,8,10,64]
3. 注意力分数：[2,8,10,10]
4. FFN升维后：[2,10,2048]

五、周测高频错题&避坑总结

💡 **本周核心避坑清单（必记）**

- **维度报错**：多头拆分/拼接、FFN升维降维、掩码shape不匹配是重灾区，严格遵循[batch, seq, d_model]格式
- **掩码失效**：掩码未扩展维度、布尔值与数值混用，导致未来信息/填充符未被屏蔽
- **梯度异常**：未做Xavier初始化、掩码极值设为-∞、学习率过高，引发梯度NaN/爆炸
- **残差错误**：残差分支加Dropout、Pre-LN顺序颠倒，导致模型不收敛
- **设备报错**：MPS设备不支持部分张量操作，需开启回退配置，掩码转为bool类型

六、本周学习总结+后续进阶规划

本周成果验收

✅ 吃透Attention模块全原理
✅ 独立手写6大核心组件
✅ 完成完整Transformer Block串联
✅ 掌握调试与避坑技巧

后续进阶学习路线

- **下一阶段**：位置编码（Positional Encoding）、词嵌入层（Embedding）
- **模型搭建**：堆叠Transformer Block，实现极简GPT解码器
- **训练实战**：构造数据集、定义损失函数、实现自回归文本生成
- **优化调参**：学习率调度、权重初始化、正则化进阶技巧

🎯 恭喜完成Attention模块周测！这是手写GPT的核心里程碑，后续将进入完整模型搭建阶段，继续加油～

（注：文档部分内容可能由 AI 生成）